Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

STRUCTURES DE DONNÉES

Chapitre 6 : Les arbres — représentation et parcours

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

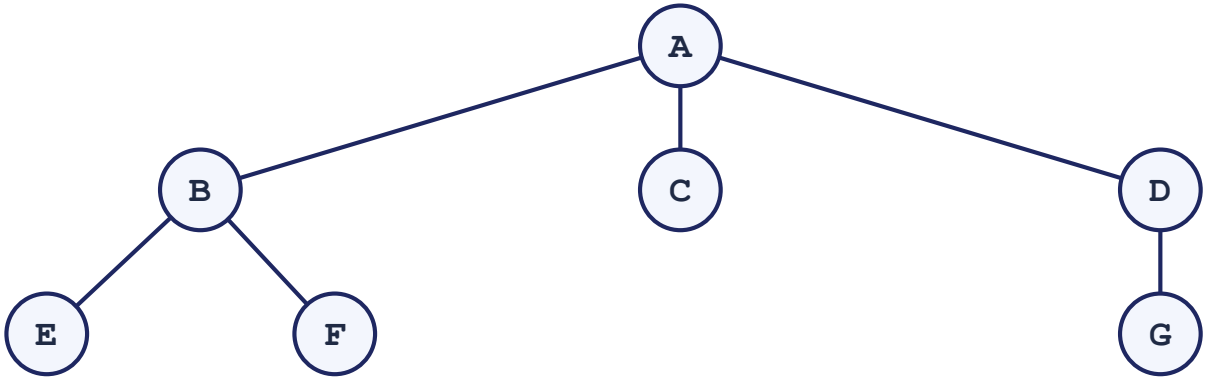
Pr. M. OUTANOUTE

Jusqu'ici, toujours une chaîne

Depuis le Chapitre 2, chaque nœud pointait vers un seul ***suivant*** (ou deux, avec ***precedent***) — toujours une *chaîne linéaire*.

Et si un nœud pouvait pointer vers plusieurs autres nœuds à la fois ?

Qu'est-ce qu'un arbre, en général ?



Une racine unique, sans parent.

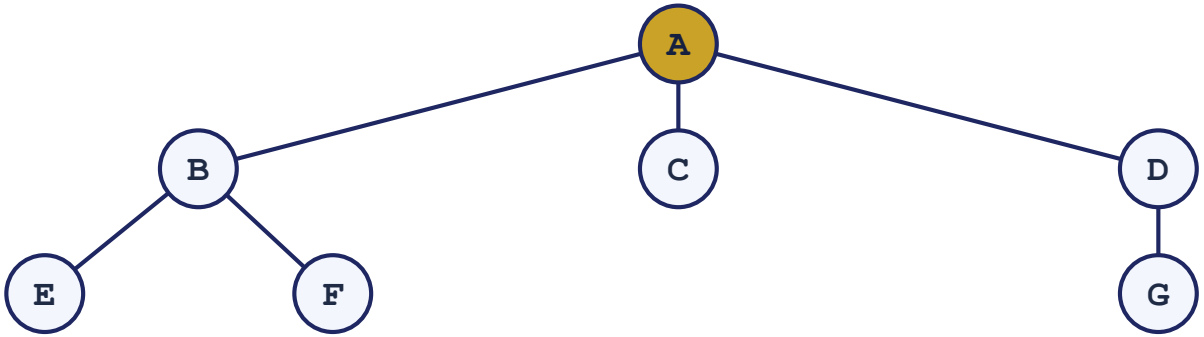
Chaque autre nœud a exactement un parent.

Un nœud peut avoir un nombre quelconque de fils.

Définition récursive :

un arbre est vide, ou une racine avec un ensemble de sous-arbres — qui sont eux-mêmes des arbres.

Le vocabulaire de base



Nœud : chaque élément de l'arbre

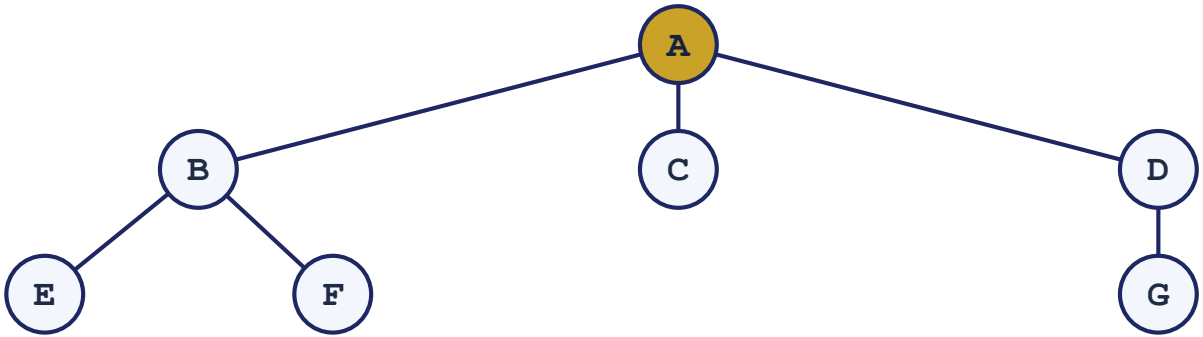
Racine : le nœud sans parent (A)

Fils : nœuds reliés en dessous (A a pour fils B, C et D)

Feuille : nœud sans aucun fils (E, F, C et G)

Sous-arbre : un nœud et tout ce qui pend de lui

Taille, et degré d'un nœud



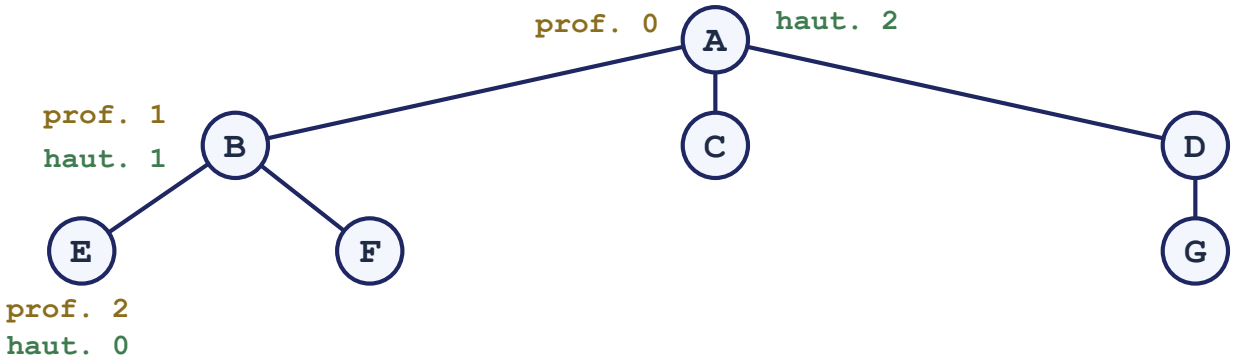
Taille (d'un arbre) : nombre total de nœuds (ici : 7 ; arbre vide -> 0)

Degré (d'un nœud) : nombre de fils, sans compter le parent

(A: 3, B: 2, D: 1, C: 0)

Degré de l'arbre : le plus grand des degrés (ici : 3)

Branche, profondeur, hauteur



Branche : chemin complet racine -> feuille (ex : A-B-E)

Profondeur(d'un nœud) : liens depuis la racine (racine: 0)

Hauteur (d'un nœud) : liens jusqu'à sa feuille la plus loin

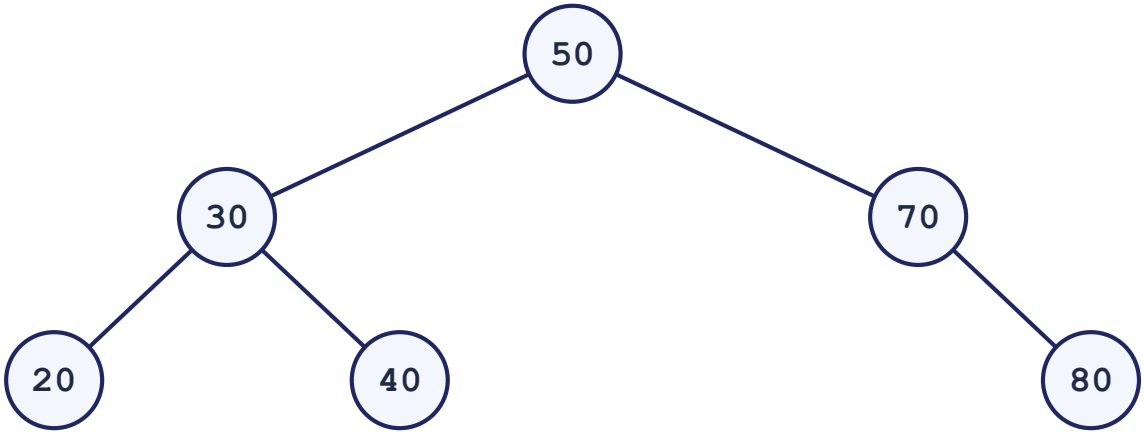
Hauteur de l'arbre : hauteur de la racine (ici : 2)

Cas limites :

- La hauteur d'un arbre composé d'un seul nœud = 0 ;
- La hauteur d'un arbre vide = -1.

Le cas particulier qui nous intéresse

Arbre binaire : chaque nœud a au plus *deux fils*, distingués par position — ***gauche*** et ***droit***.
Autrement dit : *un arbre de degré 2*.



Pourquoi ce cas particulier :
en C, deux champs nommés (**gauche**, **droit**) — plus simple qu'une liste de fils à taille variable.

La définition récursive — le cœur du chapitre

Le vocabulaire déjà posé reste identique — seul fils se précise en ***gauche*** et ***droit***.

Un arbre binaire est, par définition, soit vide, soit un nœud accompagné de deux sous-arbres — qui sont eux-mêmes des arbres binaires.

Cette définition qui se référence elle-même est exactement pourquoi les opérations sur les arbres seront naturellement récursives.

La récursion : une vraie nouveauté

Jusqu'ici, toutes les opérations utilisaient des boucles.

Sur un arbre, l'opération naturelle est une fonction qui s'appelle elle-même.

Le cas de base d'une fonction récursive sur un arbre est toujours le même :

que faire si l'arbre est vide ?

L'équivalent exact du test ***pileVide*/*fileVide*** — réévalué à chaque appel récursif.

```
typedef struct noeud {
    int valeur;
    struct noeud *gauche, *droit;
} Noeud;
Noeud *racine = NULL;
```

Trois façons de visiter le même arbre

Préfixe : racine, gauche, droit.

Infixe : gauche, racine, droit.

Postfixe : gauche, droit, racine.

Récapitulatif

- Arbre, en général : racine unique, chaque nœud a un seul parent, nombre de fils quelconque.
- Vocabulaire : nœud, racine, fils, feuille, sous-arbre, taille, degré (sans compter le parent).
- Branche = chemin racine->feuille. Profondeur, hauteur d'un nœud, hauteur de l'arbre (vide -> -1).
- Cas particulier de ce chapitre : l'arbre binaire — un arbre de degré 2 (gauche, droit).
- Définition récursive (binaire) : arbre = vide, ou nœud + deux sous-arbres qui sont eux-mêmes des arbres.
- La récursion : une vraie nouveauté — le cas de base teste toujours si le nœud est NULL.
- Trois parcours : préfixe, infixe, postfixe — l'ordre de traitement de la racine change.

Déclaration, et créer un nœud

```
typedef struct noeud {
    int valeur;
    struct noeud *gauche;
    struct noeud *droit;
} Noeud;

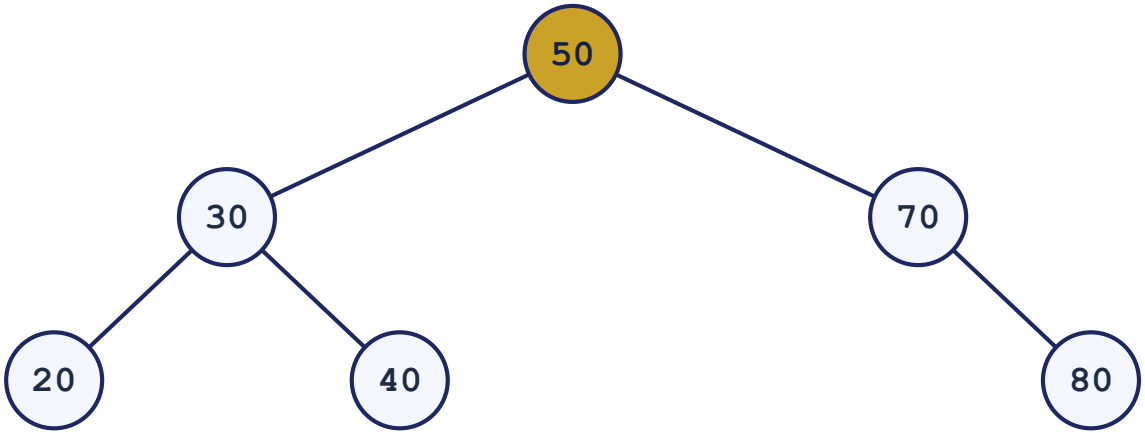
Noeud *racine = NULL;
```

1) Allouer un ***nouveau*** nœud.
2) Lui donner *sa* ***valeur***, et *ses deux enfants* à *NULL*.
3) Retourner ce nouveau nœud.

```
Noeud *creerNoeud(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->gauche = NULL;
    nouveau->droit = NULL;
    return nouveau;
}
```

Construire un arbre, à la main

```
racine = creerNoeud(50);
racine->gauche = creerNoeud(30);
racine->droit = creerNoeud(70);
racine->gauche->gauche = creerNoeud(20);
racine->gauche->droit = creerNoeud(40);
racine->droit->droit = creerNoeud(80);
```



parcoursPrefixe

1) Cas de base : si le nœud est NULL, ne rien faire.
2) Sinon, traiter la racine (l'afficher), puis parcourir le sous-arbre gauche, puis le sous-arbre droit.

```
void parcoursPrefixe(Noeud *n) {
    if (n == NULL) { return; }
    printf("%d ", n->valeur);
    parcoursPrefixe(n->gauche);
    parcoursPrefixe(n->droit);
}
```

n n’est pas la racine : chaque appel récursif porte sur un sous-arbre différent.
L'appel initial reste ***parcoursPrefixe(racine);***.

parcoursInfixe

1) Cas de base : si le nœud est NULL, ne rien faire.
2) Sinon, parcourir le sous-arbre gauche, puis traiter la racine, puis parcourir le sous-arbre droit.

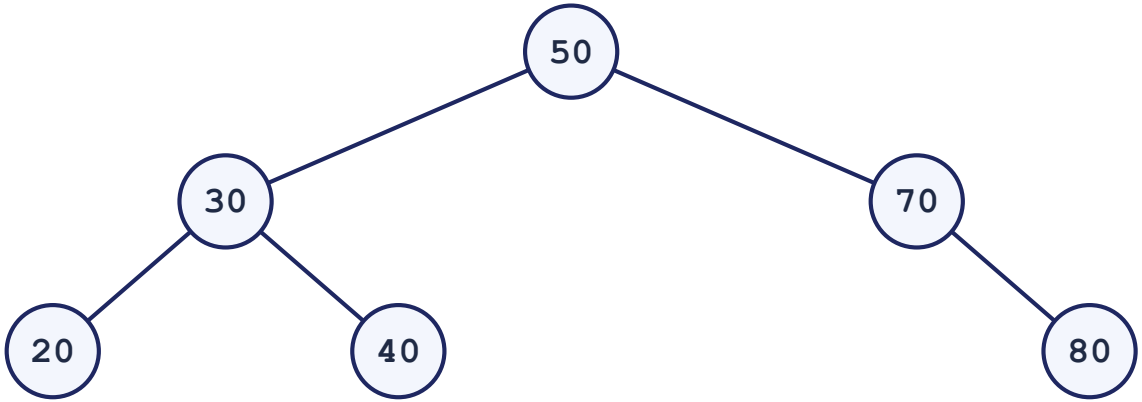
```
void parcoursInfixe(Noeud *n) {
    if (n == NULL) { return; }
    parcoursInfixe(n->gauche);
    printf("%d ", n->valeur);
    parcoursInfixe(n->droit);
}
```

parcoursPostfixe

1) Cas de base : si le nœud est NULL, ne rien faire.
2) Sinon, parcourir le sous-arbre gauche, puis le sous-arbre droit, puis traiter la racine.

```
void parcoursPostfixe(Noeud *n) {
    if (n == NULL) { return; }
    parcoursPostfixe(n->gauche);
    parcoursPostfixe(n->droit);
    printf("%d ", n->valeur);
}
```

Les trois parcours, sur le même arbre



Parcours	Résultat
Préfixe	50 30 20 40 70 80
Infixe	20 30 40 50 70 80
Postfixe	20 40 30 80 70 50

Détruire l'arbre

1) Cas de base : si le nœud est NULL, ne rien faire.
2) Détruire le sous-arbre gauche, puis le sous-arbre droit.
3) Libérer le nœud lui-même.

```
void detruireArbre(Noeud *n) {
    if (n == NULL) { return; }
    detruireArbre(n->gauche);
    detruireArbre(n->droit);
    free(n);
}
```

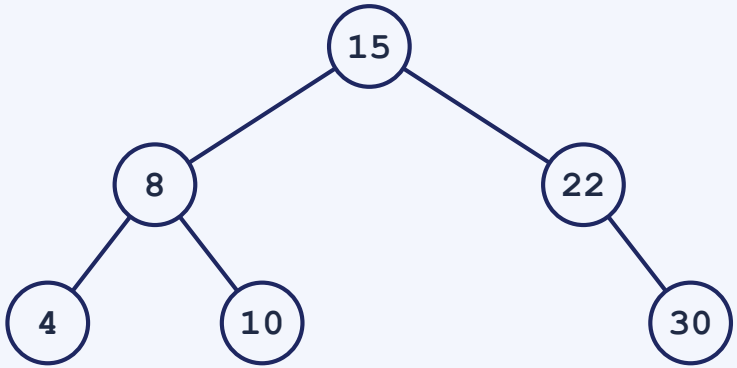
Un nœud ne peut être libéré qu'après ses deux sous-arbres
— sinon ***gauche***/***droit*** deviendraient inaccessibles, fuite mémoire.

Récapitulatif

- ***creerNoeud*** construit un nœud isolé ; l'arbre se construit à la main, par affectations.
- ***parcoursPrefixe*** : racine, gauche, droit.
- ***parcoursInfixe*** : gauche, racine , droit.
- ***parcoursPostfixe*** : gauche, droit, racine.
- Les fonctions récursives prennent un nœud en paramètre — jamais racine directement.
- ***detruireArbre*** libère les sous-arbres avant le nœud lui-même (postfixe).

Exercice d’application

Construire l'arbre suivant, à la main :

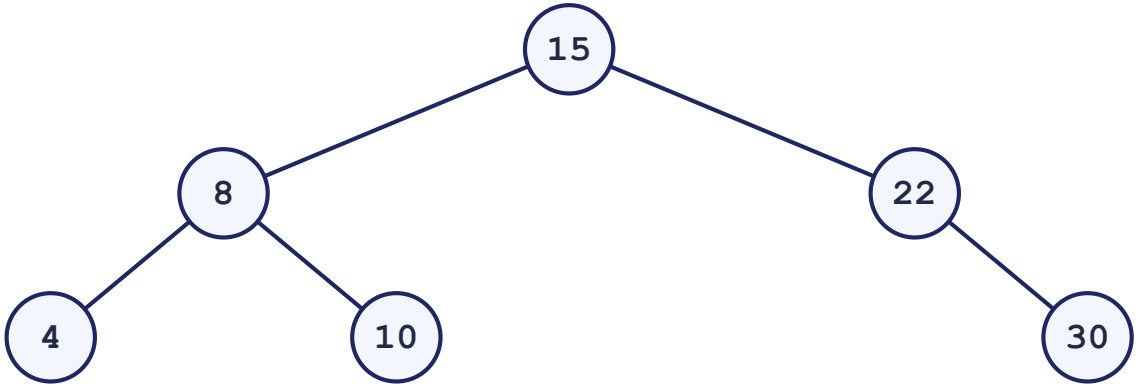


1) Écrire le code de construction (appel à ***creerNoeud*** et les affectations).
2) Dérouler ***parcoursPrefixe***, ***parcoursInfixe***, ***parcoursPostfixe*** sur cet arbre.
3) Un résultat parmi les trois vous semble-t-il particulier ? Pourquoi ?

Question 1 — le code

```
racine = creerNoeud(15);
racine->gauche = creerNoeud(8);
racine->droit = creerNoeud(22);
racine->gauche->gauche = creerNoeud(4);
racine->gauche->droit = creerNoeud(10);
racine->droit->droit = creerNoeud(30);
```

Question 2 — les trois parcours



Parcours	Résultat
Préfixe	15 8 4 10 22 30
Infixe	4 8 10 15 22 30
Postfixe	4 10 8 30 22 15

Question 3 — la remarque

Réponse attendue :

Le parcours infixe donne les valeurs dans l'ordre croissant (4, 8, 10, 15, 22, 30) — ce n'est pas un hasard.

Cet arbre respecte une règle d'ordre : chaque valeur à gauche est plus petite que son parent, chaque valeur à droite est plus grande.

C'est exactement la règle qui définit l'***arbre binaire de recherche***, sujet du Chapitre 7.